

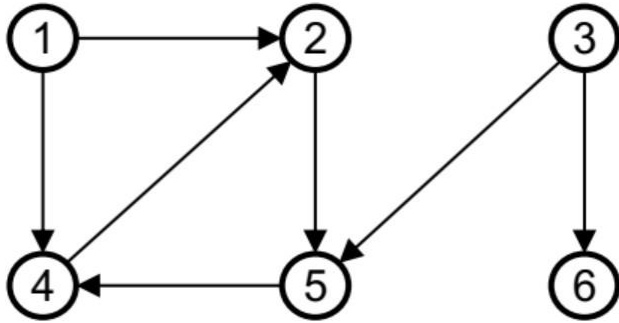
Data Structures

Lecture 22: Shortest Path

Soobin Um

Jun. 3, 2026

Recap: Definition of Graph

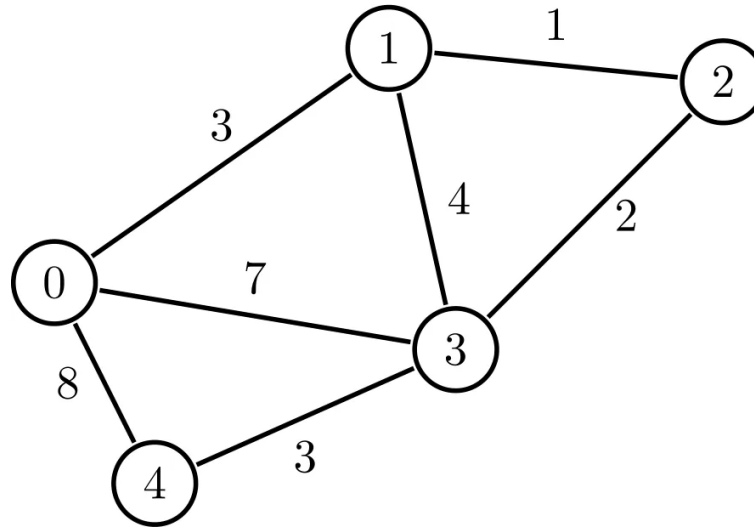


$$V = \{1,2,3,4,5,6\}$$

$$E = \{(1,2), (1,4), (2,5), (3,5), (3,6), (4,2), (5,4)\}$$

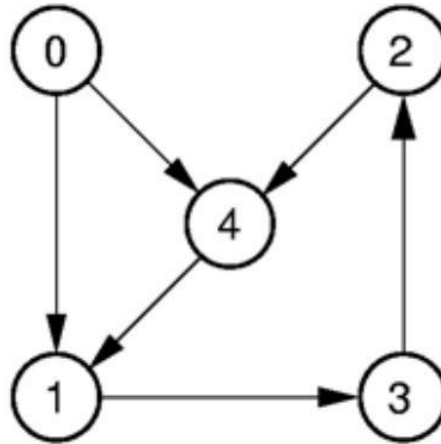
- 그래프 $G=(V, E)$ 는 다음의 두 가지 집합으로 이루어진 자료 구조
 - 정점 집합 V (Vertices)
 - 간선 집합 E (Edges)
- 정점의 개수는 $|V|$, 간선의 개수는 $|E|$ 로 표기

Recap: Weighted Graph (가중치 그래프)



- 간선마다 비용(가중치, weight)이 부여된 그래프
- 거리, 비용 등의 값을 표현할 때 사용
 - Ex) 지도 (거리), 네트워크 비용, 교통 흐름 등

Recap: Path (경로)

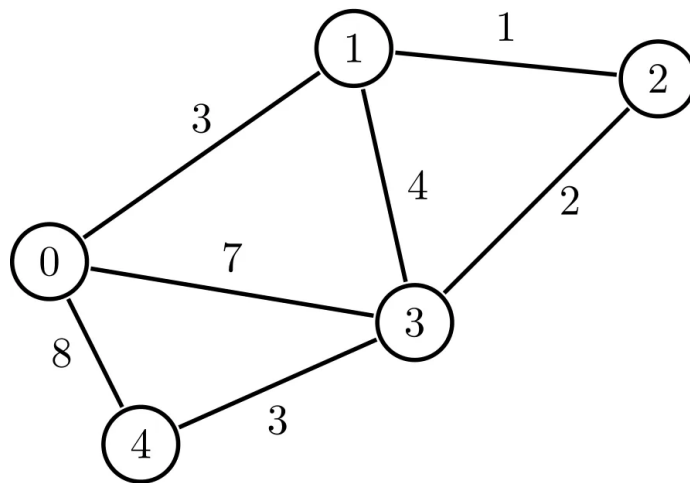


- 정점들의 나열 $v_1, v_2, \dots, v_n$ 에 대하여, 정점 v_i 와 v_{i+1} 사이에 간선이 존재하는 경우 이러한 정점들의 나열을 길이가 $n - 1$ 인 경로로 간주 (for $1 \leq i < n$)
 - 0, 4, 1, 3, 2, 4: 경로

Shortest Path Problem

Shortest Path Problem (최단 경로 문제)

- 목적
 - 단순히 최단 경로 자체를 찾기 위함
 - 또는, 최단 경로의 최단 거리 값을 얻기 위함(Ex: 네트워크 비용, 최소 지연 시간)
- 입력: 가중치가 있는 그래프 (weighted graph)
 - 정점 집합 V , 간선 집합 E
 - 각 간선은 길이 (weight) 또는 비용 (cost) 보유



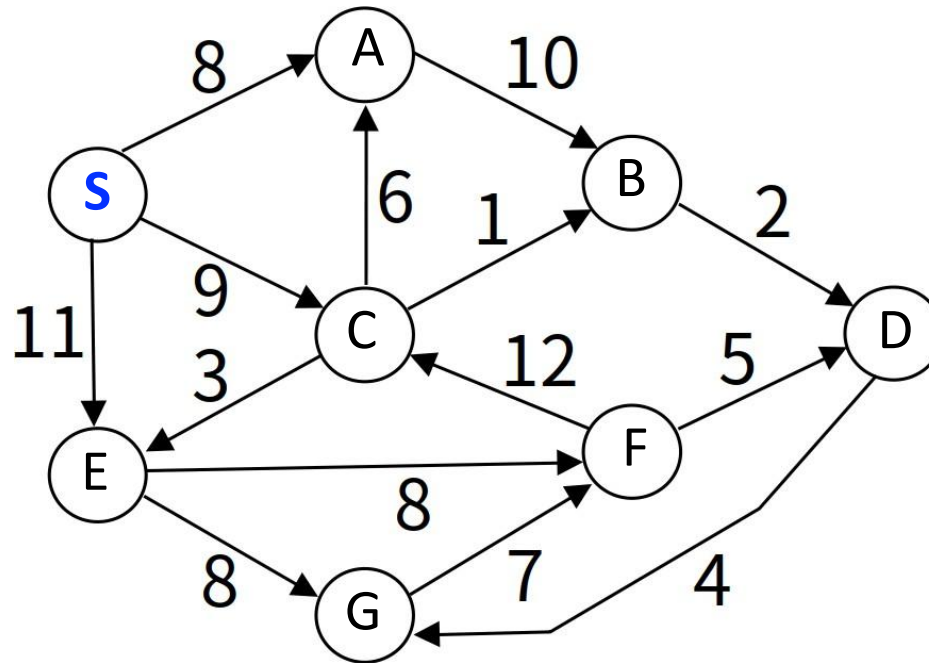
Shortest Path Problems

- **문제 (1): 특정 두 정점 사이의 최단 경로**
 - 정점 $u \rightarrow v$ 까지의 가장 짧은 경로 찾기 (예: 내비게이션 길 찾기)
- **문제 (2): 한 정점에서 모든 정점까지의 최단 경로 (Our focus)**
 - Single-Source Shortest Path (SSSP) 즉, 단일 시작점 최단 경로 문제
 - 시작 정점 s 가 주어졌을 때, s 로부터 모든 정점 v 에 대한 최단 거리 $\text{dist}[v]$ 계산
 - 대표 알고리즘: **Dijkstra's Algorithm**, Bellman-Ford Algorithm
- **문제 (3): 모든 정점 쌍 간의 최단 경로**
 - All-Pairs Shortest Paths (ASP) 즉, 전체 쌍 최단 경로 문제
 - 모든 정점 쌍 (u, v) 에 대해 쌍을 이루는 두 정점 간 최단 거리 계산
 - 대표 알고리즘: Floyd-Warshall Algorithm

Dijkstra's Algorithm

Single-Source Shortest Path (단일 시작점 최단 경로 문제)

- 시작 정점 s 가 주어졌을 때, s 로부터 모든 정점 v 에 대한 최단 거리 $\text{dist}[v]$ 계산



Setup

- S: 이미 최단 거리가 확정된 정점들의 집합
 - 초기화: $S = \emptyset$
- D: 거리 배열 (총 정점 수만큼의 사이즈)
 - $D[i]$: 현재까지 알려진 $s \rightarrow i$ 의 최단 거리 추정 값
 - 초기화: $D[s] = 0$, 나머지 정점들에 대한 거리 값은 ∞

Procedure

S: 최단 거리 정점 집합
D: 거리 배열

- **Step 1. Select Min-Distance Vertex**

- 집합 $V - S$ 중, $D[u]$ 값이 가장 작은 정점 u 를 선택
- 정점 u 를 S 에 추가 ($\rightarrow$ 최단 거리 확정)

- **Step 2. Relaxation for Each Neighbor**

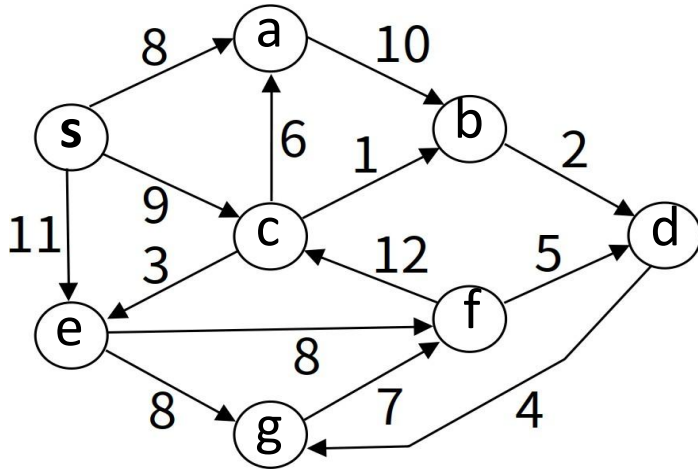
- 선택된 정점 u 의 이웃 v 에 대해, relax 연산 수행
 $\rightarrow u$ 를 거쳐 v 로 가는 경로가 더 짧으면 $D[v]$ 값을 갱신

relaxation (u, v, w):
if $d(s, v) > d(s, u) + w$:
 $d(s, v) = d(s, u) + w$
 $\text{prev}(v) = u$

- **Step 3. Termination**

- 모든 정점이 S 에 포함될 때까지 Steps 1 & 2를 반복

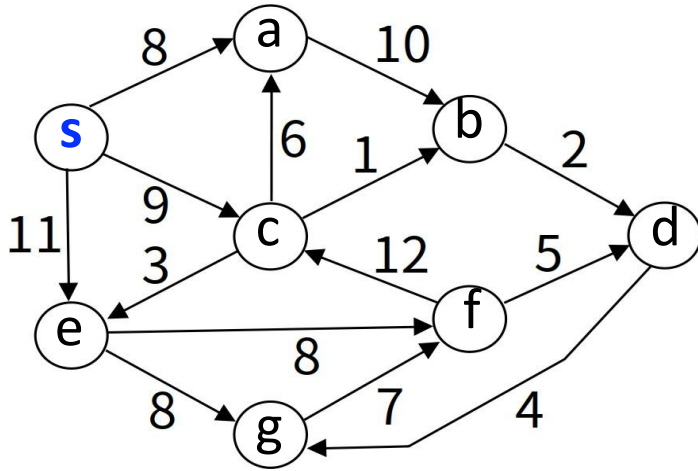
Dijkstra's Algorithm



Distance Array D

	s	a	b	c	d	e	f	g
Init.	0	∞	∞	∞	∞	∞	∞	∞
1								
2								
3								
4								
5								
6								
7								
8								

Dijkstra's Algorithm

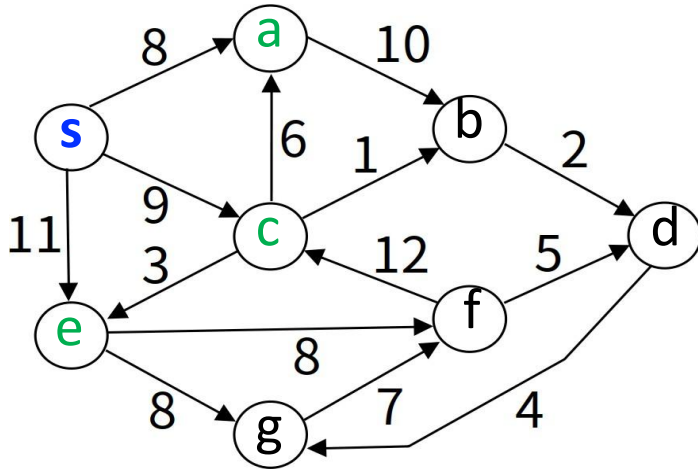


최단 거리가 확정된
정점들의 집합 S

Distance Array D

	s	a	b	c	d	e	f	g
Init.	0	∞	∞	∞	∞	∞	∞	∞
1	0							
2								
3								
4								
5								
6								
7								
8								

Dijkstra's Algorithm



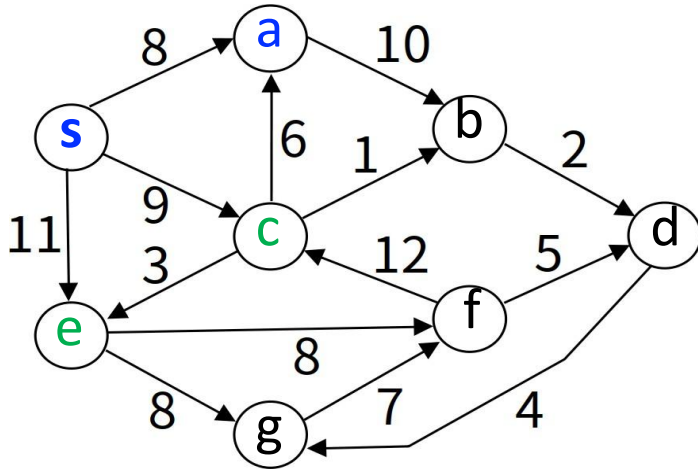
Distance Array D

	s	a	b	c	d	e	f	g
Init.	0	∞	∞	∞	∞	∞	∞	∞
1	0	8	∞	9	∞	11	∞	∞
2								
3								
4								
5								
6								
7								
8								



최단 거리가 확정된
정점들의 집합 S

Dijkstra's Algorithm

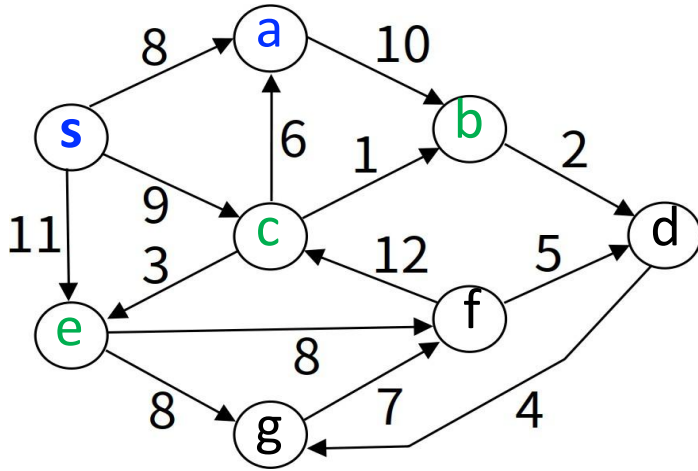


최단 거리가 확정된
정점들의 집합 S

Distance Array D

	s	a	b	c	d	e	f	g
Init.	0	∞	∞	∞	∞	∞	∞	∞
1	0	8	∞	9	∞	11	∞	∞
2	0	8						
3								
4								
5								
6								
7								
8								

Dijkstra's Algorithm

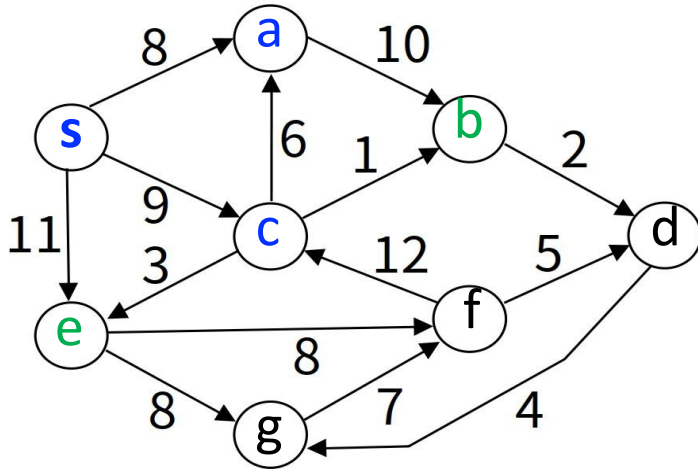


최단 거리가 확정된
정점들의 집합 S

Distance Array D

	s	a	b	c	d	e	f	g
Init.	0	∞	∞	∞	∞	∞	∞	∞
1	0	8	∞	9	∞	11	∞	∞
2	0	8	18	9	∞	11	∞	∞
3								
4								
5								
6								
7								
8								

Dijkstra's Algorithm

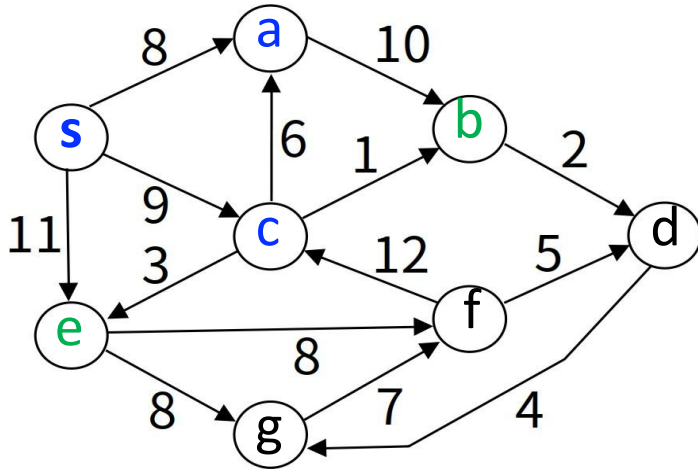


최단 거리가 확정된
정점들의 집합 S

Distance Array D

	s	a	b	c	d	e	f	g
Init.	0	∞	∞	∞	∞	∞	∞	∞
1	0	8	∞	9	∞	11	∞	∞
2	0	8	18	9	∞	11	∞	∞
3	0	8		9				
4								
5								
6								
7								
8								

Dijkstra's Algorithm



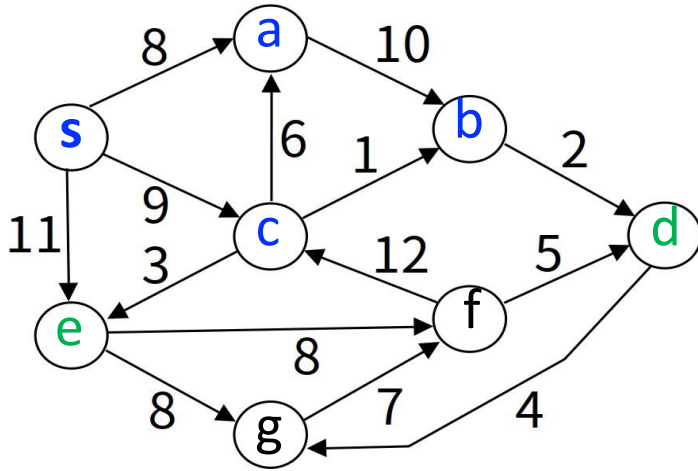
Distance Array D

	s	a	b	c	d	e	f	g
Init.	0	∞	∞	∞	∞	∞	∞	∞
1	0	8	∞	9	∞	11	∞	∞
2	0	8	18	9	∞	11	∞	∞
3	0	8	10	9	∞	11	∞	∞
4								
5								
6								
7								
8								



최단 거리가 확정된
정점들의 집합 S

Dijkstra's Algorithm



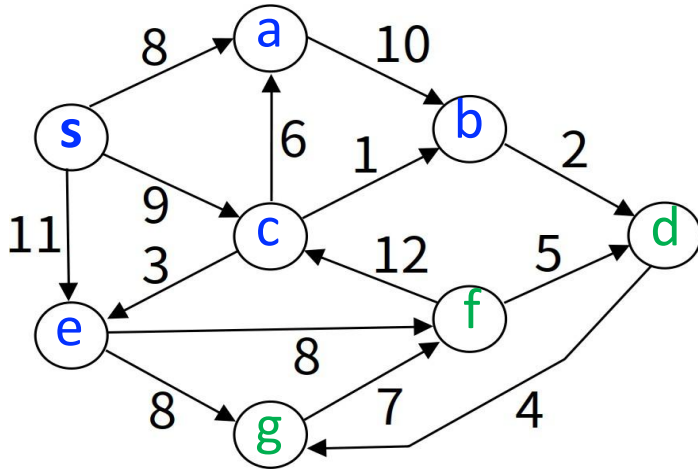
Distance Array D

	s	a	b	c	d	e	f	g
Init.	0	∞	∞	∞	∞	∞	∞	∞
1	0	8	∞	9	∞	11	∞	∞
2	0	8	18	9	∞	11	∞	∞
3	0	8	10	9	∞	11	∞	∞
4	0	8	10	9	12	11	∞	∞
5								
6								
7								
8								



최단 거리가 확정된
정점들의 집합 S

Dijkstra's Algorithm



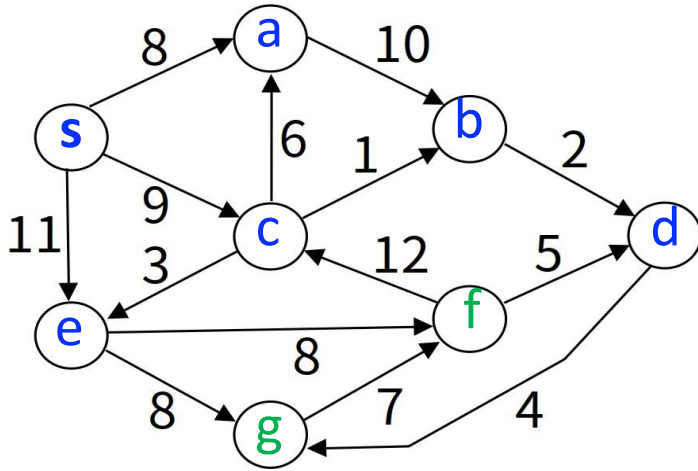
Distance Array D

	s	a	b	c	d	e	f	g
Init.	0	∞	∞	∞	∞	∞	∞	∞
1	0	8	∞	9	∞	11	∞	∞
2	0	8	18	9	∞	11	∞	∞
3	0	8	10	9	∞	11	∞	∞
4	0	8	10	9	12	11	∞	∞
5	0	8	10	9	12	11	19	19
6								
7								
8								



최단 거리가 확정된
정점들의 집합 S

Dijkstra's Algorithm



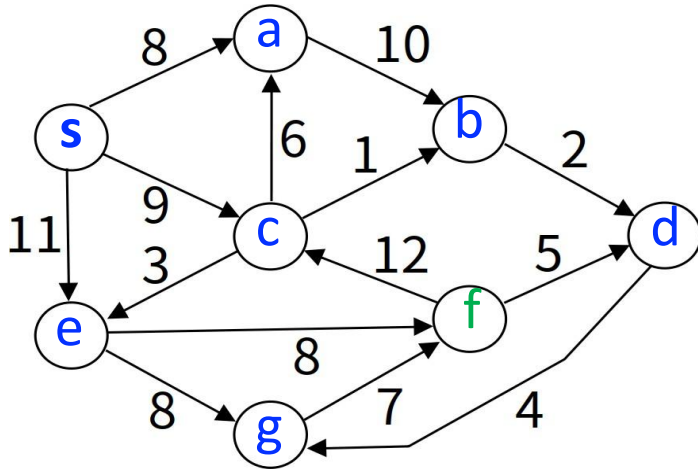
Distance Array D

	s	a	b	c	d	e	f	g
Init.	0	∞	∞	∞	∞	∞	∞	∞
1	0	8	∞	9	∞	11	∞	∞
2	0	8	18	9	∞	11	∞	∞
3	0	8	10	9	∞	11	∞	∞
4	0	8	10	9	12	11	∞	∞
5	0	8	10	9	12	11	19	19
6	0	8	10	9	12	11	19	16
7								
8								



최단 거리가 확정된
정점들의 집합 S

Dijkstra's Algorithm



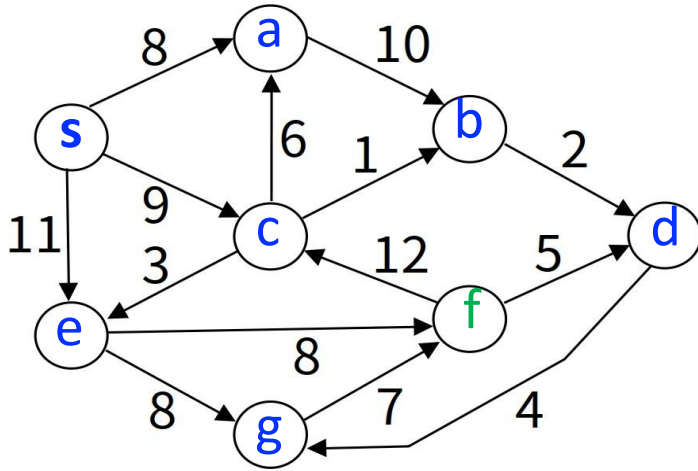
Distance Array D

	s	a	b	c	d	e	f	g
Init.	0	∞	∞	∞	∞	∞	∞	∞
1	0	8	∞	9	∞	11	∞	∞
2	0	8	18	9	∞	11	∞	∞
3	0	8	10	9	∞	11	∞	∞
4	0	8	10	9	12	11	∞	∞
5	0	8	10	9	12	11	19	19
6	0	8	10	9	12	11	19	16
7	0	8	10	9	12	11	19	16
8								



최단 거리가 확정된
정점들의 집합 S

Dijkstra's Algorithm



Distance Array D

	s	a	b	c	d	e	f	g
Init.	0	∞	∞	∞	∞	∞	∞	∞
1	0	8	∞	9	∞	11	∞	∞
2	0	8	18	9	∞	11	∞	∞
3	0	8	10	9	∞	11	∞	∞
4	0	8	10	9	12	11	∞	∞
5	0	8	10	9	12	11	19	19
6	0	8	10	9	12	11	19	16
7	0	8	10	9	12	11	19	16
8	0	8	10	9	12	11	19	16



최단 거리가 확정된
정점들의 집합 S

Pseudocode

```
function Dijkstra(G, s):  
    S = {s}  
    initialize an array D of size |V| with infinity  
    initialize an array prev of size |V|  
  
    D[s] = 0  
    while S != V:  
        u = minDistVertex(V-S, D)  
        add u to S  
        for v in G.neighbors(u):  
            if v is not in S and D[u] + w(u,v) < D[v]:  
                D[v] = D[u] + w(u,v)  
                prev[v] = u
```


minDistVertex()

- “How to pick the next-closest vertex?”
 - 아직 방문 되지 않은 정점들 중, D 값이 가장 작은 정점 u 를 선택 (→ 다익스트라 핵심 단계)
 - 구현 방식에 따라 전체 시간 복잡도가 달라질 수 있음
- **방법 1: Distance Table (배열) 스캔**
 - 매 단계마다 거리 배열 D 를 앞에서부터 끝까지 선형 탐색
 - 방문 되지 않은 정점 중 D 값이 최소인 정점 찾기
 - **비용:** $O(|V|^2 + |E|) = O(|V|^2)$
 - 정점 v 번 선택 후, 매번 $O(|V|)$ 회의 검색 수행 → $O(|V|^2)$
 - $|E|$ 회의 relaxation 발생 → $O(|E|)$
- **방법 2: Min-Heap 사용 (사실상 표준)**
 - 방문 되지 않은 정점들을 min-heap (priority queue)에 저장
 - D 값이 가장 작은 정점을 바로 추출 가능
 - **비용:** $O((|V| + |E|) \cdot \log |V|)$
 - 초기 buildHeap 1회 → $O(|V|)$ (무시됨)
 - 매 정점에 대해서 extract-min 필요 → $O(|V| \cdot \log |V|)$
 - 간선마다 relaxation 발생 시 heap 업데이트 필요 → $O(|E| \cdot \log |V|)$

Look ahead

Minimum Spanning Tree